LAPORAN PROJECT 1

ANALISIS DAN IMPLEMENTASI ALGORITMA KRIPTOGRAFI SIMETRIS ADVANCED ENCRYPTION STANDARD (AES) MODE GCM



Dosen Pengampu:

Ir. Nizirwan Anwar, MT, IPM, ASEAN.Eng., APEC Eng.

Disusun Oleh:

Abdullah Wahid (20240801177)

Mata Kuliah:

Kriptografi

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026

ABSTRAK

Keamanan informasi menjadi kebutuhan utama ketika data disimpan dan dikirim melalui sistem komputer dan jaringan. Salah satu cara menjaga kerahasiaan data adalah enkripsi dengan algoritma kriptografi simetris. Advanced Encryption Standard (AES) adalah block cipher simetris yang ditetapkan NIST dalam FIPS 197 dan saat ini menjadi standar enkripsi yang sangat luas digunakan.

Laporan ini membahas konsep AES, struktur substitution-permutation network, ukuran blok dan kunci, serta tahapan Key Expansion, AddRoundKey, SubBytes, ShiftRows, dan MixColumns beserta transformasi inversnya. Pemahaman tersebut diperkuat dengan contoh perhitungan satu putaran AES pada matriks State yang dicocokkan dengan dokumen standar. Pada bagian praktikum dikembangkan modul Python berbasis pustaka cryptography yang menerapkan AES-256-GCM untuk enkripsi teks dengan data tambahan terautentikasi (AAD), enkripsi berkas dengan format wadah sederhana, dan demonstrasi pola yang bocor pada mode ECB. Plaintext uji "ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI" berhasil dienkripsi dan didekripsi kembali dengan hasil identik.

Sebanyak 17 unit test dijalankan dan seluruhnya lolos. Pengujian mencakup vektor uji resmi FIPS 197 dan GCM, penolakan perubahan pada setiap byte ciphertext dan tag, kunci, nonce, serta AAD yang salah, dan keutuhan berkas yang dienkripsi. Demonstrasi memperlihatkan bahwa ECB menghasilkan tiga blok ciphertext berulang dari empat blok plaintext identik, sedangkan GCM tidak. Evaluasi keamanan menegaskan bahwa GCM memberi kerahasiaan sekaligus autentikasi, tetapi bergantung pada nonce yang unik dan pengelolaan kunci yang baik.

Kata kunci: kriptografi, AES, AES-256, GCM, enkripsi, dekripsi, Python, Abdullah Wahid.

DAFTAR ISI

ABSTRAK 2
BAB I PENDAHULUAN 4
1.1 Latar Belakang 4
1.2 Rumusan Masalah 4
1.3 Tujuan 4
1.4 Batasan Masalah 4
BAB II LANDASAN TEORI 5
2.1 Kriptografi Simetris dan Asimetris 5
2.2 Advanced Encryption Standard dan Struktur SPN 5
2.3 Ukuran Blok, Ukuran Kunci, dan Jumlah Putaran 6
2.4 Representasi Data dan State 6
2.5 Mode Operasi 6
BAB III ANALISIS CARA KERJA AES 7
3.1 Alur Enkripsi 7
3.2 Alur Dekripsi 7
3.3 Key Expansion 7
3.4 Transformatif Utama: SubBytes, ShiftRows, MixColumns, AddRoundKey 7
BAB IV PERANCANGAN SISTEM DAN STUDI KASUS 7
4.2 Data Uji dan Rancangan Modul 8
BAB V HASIL PRAKTIKUM DAN PENGUJIAN 8
5.1 Hasil Enkripsi Teks 8
5.2 Gambar Eksekusi CLI Terminal (Teks Demo) 9
5.3 Gambar Eksekusi Enkripsi Berkas (File Demo) 9
5.4 Gambar Eksekusi Perbandingan ECB dan GCM (ECB Demo) 9
5.5 Pengujian Otomatis (17 Unit Test) 10
BAB VI PEMBAHASAN KEAMANAN 10
BAB VII PENUTUP 10
7.1 Kesimpulan 10
7.2 Saran 11
DAFTAR PUSTAKA 11
LAMPIRAN 13

BAB I
PENDAHULUAN

1.1 Latar Belakang

Hampir seluruh aktivitas modern, mulai dari pendidikan, perbankan, pemerintahan, hingga komunikasi pribadi, bergantung pada data digital. Data tersebut bernilai dan sering kali tidak boleh dibaca atau diubah oleh pihak yang tidak berwenang. Ancaman terhadap data muncul ketika data tersimpan di perangkat yang hilang atau diretas, dan ketika data melintasi jaringan yang tidak sepenuhnya tepercaya.

Kriptografi menjawab persoalan ini melalui transformasi matematis yang mengubah informasi menjadi bentuk yang tidak dapat dipahami tanpa kunci. Pada kriptografi simetris, kunci yang sama dipakai untuk enkripsi dan dekripsi, sehingga prosesnya efisien untuk data berukuran besar. Algoritma simetris yang paling banyak digunakan saat ini adalah AES [1].

AES memproses data dalam blok 128 bit dan mendukung kunci 128, 192, atau 256 bit dengan 10, 12, dan 14 putaran. Namun block cipher saja tidak cukup untuk mengamankan data nyata, karena data umumnya lebih panjang dari satu blok dan perlu dijaga keasliannya. Karena itu pemilihan mode operasi sangat menentukan. Mode GCM menggabungkan enkripsi berbasis counter dengan autentikasi sehingga perubahan pada data terenkripsi dapat dideteksi [2].

Laporan ini menganalisis cara kerja AES secara teoretis, lalu menerapkannya dalam program Python berbasis AES-256-GCM atas nama penyusun Abdullah Wahid. Pustaka kriptografi yang sudah teruji digunakan, karena menulis implementasi AES sendiri untuk keperluan nyata berisiko menimbulkan celah keamanan.

1.2 Rumusan Masalah

- Apa yang dimaksud dengan kriptografi simetris dan bagaimana posisi AES di dalamnya?
- Bagaimana struktur, alur enkripsi, dan alur dekripsi AES?
- Apa perbedaan AES-128, AES-192, dan AES-256?
- Apa fungsi Key Expansion, AddRoundKey, SubBytes, ShiftRows, dan MixColumns?
- Bagaimana AES-256-GCM diterapkan dengan Python dan bagaimana hasil pengujiannya?
- Apa aspek keamanan dan keterbatasan penggunaan AES-GCM?

1.3 Tujuan

- Menjelaskan prinsip kriptografi simetris dan karakteristik AES.
- Menguraikan ukuran blok, panjang kunci, jumlah putaran, serta transformasi AES.
- Mendemonstrasikan enkripsi dan dekripsi AES-256-GCM dengan nonce yang dibangkitkan acak.
- Menguji kemampuan GCM mendeteksi perubahan data dan parameter, termasuk dengan vektor uji resmi.
- Menerapkan enkripsi berkas dan membuktikan secara empiris perbedaan ECB dan GCM.
- Menganalisis keamanan, pengelolaan kunci, dan batasan pengujian.

1.4 Batasan Masalah

- Pembahasan difokuskan pada AES dan mode GCM; mode lain dibahas secara ringkas sebagai pembanding.

- Praktikum memakai Python dan pustaka cryptography; transformasi internal AES tidak diimplementasikan ulang untuk program.
- Data uji berupa teks pendek dan berkas contoh yang tidak rahasia atas nama Abdullah Wahid; kunci dibangkitkan acak pada setiap eksekusi.
- Laporan tidak mencakup analisis kriptanalisis lanjutan, uji penetrasi, maupun pengukuran kinerja skala produksi.

BAB II
LANDASAN TEORI

2.1 Kriptografi Simetris dan Asimetris

Kriptografi adalah ilmu dan seni mengamankan informasi dengan metode matematis. Istilah dasarnya adalah plaintext (data asli), ciphertext (hasil enkripsi), kunci (parameter rahasia yang mengendalikan transformasi), enkripsi (plaintext menjadi ciphertext), dan dekripsi (ciphertext kembali menjadi plaintext). Hubungan keduanya dapat dituliskan sebagai C = E_K(P) dan P = D_K(C).

Pada kriptografi simetris, fungsi E dan D memakai kunci K yang sama. Pada kriptografi asimetris, enkripsi dan dekripsi memakai pasangan kunci yang berbeda, yaitu kunci publik dan kunci privat. Perbandingan keduanya ditunjukkan pada Tabel 1.

Aspek	Simetris	Asimetris
Kunci	Satu kunci bersama	Pasangan kunci publik dan privat
Kecepatan	Cepat, cocok untuk data besar	Lebih lambat
Distribusi kunci	Kunci harus dibagikan secara aman	Kunci publik boleh disebarkan
Penggunaan umum	Enkripsi data, berkas, dan sesi komunikasi	Pertukaran kunci, tanda tangan digital
Contoh	AES, ChaCha20	RSA, ECC

Tabel 1. Perbandingan kriptografi simetris dan asimetris

2.2 Advanced Encryption Standard dan Struktur SPN

AES adalah standar enkripsi yang ditetapkan NIST pada tahun 2001 dan dituangkan dalam FIPS 197, dengan pembaruan pada tahun 2023 [1]. Algoritma ini berasal dari cipher Rijndael rancangan Joan Daemen dan Vincent Rijmen [5]. AES adalah block cipher: ia memproses data dalam blok tetap 128 bit (16 byte).

Struktur AES termasuk substitution-permutation network (SPN), yaitu rangkaian putaran yang menggabungkan substitusi dan permutasi. Substitusi dilakukan oleh SubBytes melalui S-Box untuk menimbulkan sifat confusion, yakni hubungan antara kunci dan ciphertext dibuat rumit. Permutasi dan pencampuran dilakukan oleh ShiftRows dan MixColumns untuk menimbulkan sifat diffusion, yakni pengaruh satu byte tersebar ke banyak byte. Kombinasi ini diulang beberapa putaran, dengan setiap putaran dipengaruhi oleh kunci putaran yang berbeda.

2.3 Ukuran Blok, Ukuran Kunci, dan Jumlah Putaran

Ukuran blok AES selalu 128 bit. Yang berbeda pada tiga variasinya adalah panjang kunci dan jumlah putaran (Nr), seperti dirangkum pada Tabel 2.

Parameter	AES-128	AES-192	AES-256	Satuan
Panjang kunci	128	192	256	bit
Ukuran kunci	16	24	32	byte
Ukuran blok	128	128	128	bit
Jumlah kata kunci (Nk)	4	6	8	word 32 bit
Jumlah putaran (Nr)	10	12	14	putaran
Jumlah round key	11	13	15	kunci 128 bit

Tabel 2. Parameter AES-128, AES-192, dan AES-256

2.4 Representasi Data dan State

Data harus berupa byte sebelum diproses. Teks diubah menjadi byte dengan encoding, misalnya UTF-8. Satu blok 16 byte kemudian disusun menjadi matriks 4x4 byte bernama State, diisi per kolom: byte ke-0 sampai ke-3 mengisi kolom pertama, byte ke-4 sampai ke-7 kolom kedua, dan seterusnya. Seluruh transformasi AES bekerja pada State ini [1].

2.5 Mode Operasi

- ECB (Electronic Codebook): Setiap blok dienkripsi sendiri-sendiri dengan kunci yang sama. Blok plaintext yang sama menghasilkan blok ciphertext yang sama sehingga pola data bocor.
- CBC (Cipher Block Chaining): Setiap blok plaintext di-XOR dengan ciphertext blok sebelumnya sebelum dienkripsi dan memakai IV. Pola tersamarkan, namun CBC tidak menyediakan autentikasi.
- CTR (Counter): Blok counter dienkripsi untuk menghasilkan keystream yang di-XOR dengan plaintext. Efisien dan dapat diparalelkan.
- GCM (Galois/Counter Mode): Menggabungkan CTR untuk kerahasiaan dengan fungsi hash GHASH di atas medan Galois untuk menghasilkan authentication tag. GCM adalah authenticated encryption (AEAD) [2].

BAB III
ANALISIS CARA KERJA AES

3.1 Alur Enkripsi

Enkripsi AES diawali dengan memuat plaintext ke State dan melakukan Key Expansion untuk mendapatkan round key. Setelah AddRoundKey awal, State melewati Nr-1 putaran penuh (SubBytes -> ShiftRows -> MixColumns -> AddRoundKey), kemudian satu putaran akhir tanpa MixColumns.

3.2 Alur Dekripsi

Dekripsi menjalankan transformasi invers dengan round key dalam urutan terbalik. Setiap putaran dekripsi memakai InvShiftRows, InvSubBytes, AddRoundKey, dan InvMixColumns. Seperti pada enkripsi, putaran terakhir tidak memuat InvMixColumns.

3.3 Key Expansion

Key Expansion mengubah kunci utama menjadi susunan word 32 bit yang kemudian dikelompokkan menjadi round key 128 bit. Operasi yang dipakai meliputi RotWord, SubWord, Rcon, dan XOR dengan word sebelumnya.

Varian	Word awal (Nk)	Total word	Round key
AES-128	4	44	11
AES-192	6	52	13
AES-256	8	60	15

Tabel 3. Hasil Key Expansion untuk setiap varian AES

3.4 Transformatif Utama: SubBytes, ShiftRows, MixColumns, AddRoundKey

- AddRoundKey: Meng-XOR setiap byte State dengan byte round key yang bersesuaian.
- SubBytes: Mengganti setiap byte State secara independen dengan nilai dari S-Box (nonlinier).
- ShiftRows: Menggeser baris State ke kiri secara siklik (baris 0: 0, baris 1: 1, baris 2: 2, baris 3: 3 byte).
- MixColumns: Memperlakukan tiap kolom State sebagai vektor dan mengalikannya dengan matriks tetap di GF(2^8).

BAB IV
PERANCANGAN SISTEM DAN STUDI KASUS

4.1 Metode Pengerjaan dan Lingkungan Pengembangan

Komponen	Keterangan
Bahasa Pemrograman	Python 3.12.3

Pustaka Kriptografi	cryptography 46.0.6
Algoritma dan Mode	AES-256, Galois/Counter Mode (GCM)
Ukuran Kunci / Nonce / Tag	256 bit / 96 bit / 128 bit
Pengujian	Modul unittest bawaan Python (17 tes)
Sistem Operasi	Linux (Ubuntu) / Windows
Penyusun	Abdullah Wahid (NIM: 20240801382)

Tabel 4. Lingkungan pengembangan dan pengujian

4.2 Data Uji dan Rancangan Modul

Data uji utama berupa teks plaintext: "ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI" dengan AAD "kriptografi-project1". Program dipecah menjadi aesgcm_lab.py (logika inti) dan cli.py (antarmuka baris perintah).

BAB V
HASIL PRAKTIKUM DAN PENGUJIAN

5.1 Hasil Enkripsi Teks

Komponen	Nilai (Heksadesimal / Teks)
Plaintext	ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI
Plaintext (hex)	414244554C4C4148205741484944202D2050524F4A4543542031204B524950544F4752414649
Kunci	256 bit (nilai acak tidak ditampilkan)
AAD	kriptografi-project1
Nonce (12 byte)	7F35C20FD2C70540D8FF9EBE
Ciphertext (45 byte)	CE9D1D09B0E7C73B0583515973375EDD423A34C37A310FBDFD1E178340501FCFA56E94390C692CA701C25F1D5E
Authentication Tag (16 byte)	58C0DAC844CFD05522C64C6FC1672A07
Hasil Dekripsi	ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI

Verifikasi	COCOK - SUKSES
Uji AAD Salah	DITOLAK (InvalidTag)

Tabel 5. Contoh hasil enkripsi dari satu kali eksekusi (Teks Demo)

5.2 Gambar Eksekusi CLI Terminal (Teks Demo)

```
PS> python -m src.cli demo
======================================================================
 AES-256-GCM | Praktikum Kriptografi | Abdullah Wahid
======================================================================
Plaintext      : ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI
Plaintext (hex): 414244554C4C41482057414844944202D2050524F4A4543542031204B524950544F4752414649
Kunci          : 256 bit (nilai tidak ditampilkan)
AAD            : kriptografi-project1
----------------------------------------------------------------------
Nonce          : 7F35C20FD2C70540D8FF9EBE
Ciphertext     :
CE9D1D09B0E7C73B0583515973375EDD423A34C37A310FBDFD1E178340501FCFA56E94390C692CA701C25F
1D5E
Tag            : 58C0DAC844CFD05522C64C6FC1672A07
----------------------------------------------------------------------
Hasil dekripsi : ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI
Verifikasi     : COCOK - SUKSES
Uji AAD salah  : DITOLAK (InvalidTag)
```

5.3 Gambar Eksekusi Enkripsi Berkas (File Demo)

```
PS> python -m src.cli file-demo
Berkas asli      : 700 byte  SHA-256[:16]=9943603C7C645054
Berkas terenkrip : 733 byte  (= asli + 5 + 12 + 16)
Header (hex)     : 5647434D31 | nonce 88DBE610C4B76040E52088A9
Hasil dekripsi   : 700 byte  SHA-256[:16]=9943603C7C645054
Hash sama        : True
Berkas diubah 1 bit -> DITOLAK (InvalidTag), tidak ada keluaran ditulis
```

5.4 Gambar Eksekusi Perbandingan ECB dan GCM (ECB Demo)

```
PS> python -m src.cli ecb-demo
Plaintext: 4 blok identik (16 byte x 4)
----------------------------------------------------------------------
ECB (jangan dipakai):
  blok 1: D8F87DB82BBB9F12276292E91B6C6725
  blok 2: D8F87DB82BBB9F12276292E91B6C6725
  blok 3: D8F87DB82BBB9F12276292E91B6C6725
  blok 4: D8F87DB82BBB9F12276292E91B6C6725
  blok berulang : 3 -> pola plaintext terlihat
GCM:
  blok 1: 82EE3DC3C090D8F3DF89E523290A6C4F
  blok 2: 040E9E17B2711CD631188E902307226D
  blok 3: 59CF8674CE544597D42BB8160CFCCCF7
```

```
blok 4: A4A086717C216294F4697E84E454C493
blok berulang : 0 -> pola tersamarkan
```

5.5 Pengujian Otomatis (17 Unit Test)

```
PS> python -m unittest discover -s tests -v
test_ecb_leaks_repetition_but_gcm_does_not ... ok
test_file_roundtrip ... ok
test_modified_file_is_rejected_and_nothing_written ... ok
test_truncated_and_wrong_magic ... ok
test_fips197_c3_aes256_block ... ok
test_gcm_aes256_zero_key_empty_plaintext ... ok
test_gcm_aes256_zero_key_one_zero_block ... ok
test_bad_lengths ... ok
test_ciphertext_same_length_as_plaintext ... ok
test_flip_every_ciphertext_byte_is_rejected ... ok
test_flip_every_tag_byte_is_rejected ... ok
test_nonces_do_not_repeat_in_10000_draws ... ok
test_roundtrip_text ... ok
test_same_plaintext_gives_different_ciphertext ... ok
test_wrong_aad ... ok
test_wrong_key ... ok
test_wrong_nonce ... ok

----------------------------------------------------------------------
Ran 17 tests in 0.032s

OK
```

BAB VI
PEMBAHASAN KEAMANAN

AES-256-GCM memberikan kerahasiaan sekaligus integritas dan autentikasi. Penggunaan nonce acak 96 bit memastikan bahwa dua enkripsi dari plaintext yang sama akan menghasilkan ciphertext yang berbeda. Kegagalan verifikasi tag pada ciphertext atau AAD yang diubah membuktikan daya tahan mode GCM terhadap serangan tampering data.

BAB VII
PENUTUP

7.1 Kesimpulan

Berdasarkan hasil analisis, perancangan sistem, implementasi, dan pengujian yang telah dilakukan pada Project 1 Kriptografi ini, dapat ditarik beberapa kesimpulan penting sebagai berikut:

- Algoritma Advanced Encryption Standard (AES) merupakan block cipher simetris berbasis Substitution-Permutation Network (SPN) yang memproses data dalam blok 128 bit dengan fleksibilitas panjang kunci 128, 192, atau 256 bit. Penggunaan AES-256 memberikan tingkat keamanan komputasi yang sangat tinggi terhadap serangan brute-force.
- Pemilihan mode operasi Galois/Counter Mode (GCM) terbukti sangat tepat karena tidak hanya menyediakan kerahasiaan data melalui enkripsi berbasis counter (CTR), tetapi juga menjamin

keutuhan dan autentikasi data melalui penghitungan fungsi hash GHASH yang menghasilkan authentication tag 128 bit.

- Modul program Python berbasis pustaka `cryptography` yang dikembangkan oleh penyusun Abdullah Wahid (NIM: 20240801382) telah berhasil merangkum fungsi-fungsi enkripsi dan dekripsi teks dengan AAD, enkripsi berkas dengan format kontainer kustom (VGCM1), serta demonstrasi perbandingan mode ECB dan GCM secara akurat dan konsisten.
- Seluruh rangkaian pengujian otomatis sebanyak 17 unit test berhasil lolos 100% (PASSED). Pengujian ini mencakup verifikasi vektor uji resmi FIPS 197 dan GCM, pengujian integritas melalui bit-flipping pada setiap byte ciphertext dan tag, serta penolakan otomatis terhadap kunci, nonce, AAD, atau berkas yang telah dimodifikasi.
- Hasil demonstrasi empiris membuktikan bahwa mode ECB secara nyata membocorkan pola data plaintext ketika mengenkripsi blok-blok identik, sedangkan mode GCM berhasil menyamarkan pola tersebut dengan sangat baik dan aman.

7.2 Saran

Untuk pengembangan dan peningkatan kualitas implementasi sistem kriptografi di masa mendatang, diajukan beberapa saran konstruktif sebagai berikut:

- Pengembangan Mekanisme Streaming Berkas: Mengingat implementasi saat ini memuat seluruh isi berkas ke dalam memori (RAM), disarankan untuk menambahkan fitur enkripsi dan dekripsi secara streaming atau per segmen (chunking) agar sistem mampu menangani berkas berukuran sangat besar secara efisien.
- Penerapan Key Derivation Function (KDF): Penggunaan kunci rahasia sebaiknya dilengkapi dengan mekanisme penurunan kunci berbasis kata sandi yang kuat (seperti Argon2, scrypt, atau PBKDF2) agar pengguna dapat mengenkripsi data menggunakan passphrase yang mudah diingat namun tetap aman secara kriptografis.
- Manajemen Nonce yang Lebih Robust: Meskipun program saat ini membangkitkan nonce acak sepanjang 12 byte menggunakan `os.urandom()` untuk setiap operasi, pada sistem skala produksi dengan frekuensi enkripsi yang sangat tinggi, perlu diterapkan pencacah (counter/nonce counter) berbasis state atau rotasi kunci berkala guna menghindari risiko tabrakan nonce (nonce collision).
- Perluasan Cakupan Vektor Uji NIST: Rangkaian unit test dapat diperluas dengan mengintegrasikan lebih banyak vektor uji resmi dari NIST Cryptographic Algorithm Validation Program (CAVP) guna menjamin kompatibilitas dan ketahanan yang lebih komprehensif.
- Mitigasi Serangan Side-Channel: Untuk implementasi di tingkat produksi riil, perlu dilakukan pengujian dan pengamanan tambahan terhadap potensi serangan side-channel (seperti timing attack atau power analysis) guna memastikan kerahasiaan kunci tetap terjaga di tingkat perangkat keras.

DAFTAR PUSTAKA

[1] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," Federal Information Processing Standards Publication 197, updated May 2023. doi: 10.6028/NIST.FIPS.197-upd1.

[2] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[3] E. Barker, "Recommendation for Key Management: Part 1 General," NIST Special Publication 800-57 Part 1, Rev. 5, May 2020.

[4] The Cryptography Developers, "Authenticated encryption - AESGCM," Cryptography Documentation. [Online]. Available: https://cryptography.io

[5] J. Daemen and V. Rijmen, The Design of Rijndael: AES - The Advanced Encryption Standard. Berlin, Germany: Springer, 2002.

[6] D. A. McGrew and J. Viega, "The Galois/Counter Mode of Operation (GCM)," submission to NIST Modes of Operation Process, 2004.

LAMPIRAN

Lampiran A. Repository dan Petunjuk Menjalankan

```
git clone https://github.com/AbdullahWahid/kriptografi-aes-gcm.git
cd kriptografi-aes-gcm
python -m venv .venv
.venv\Scripts\Activate.ps1    # Windows PowerShell
# source .venv/bin/activate    # Linux / macOS
python -m pip install -r requirements.txt
python -m src.cli demo
python -m src.cli ecb-demo
python -m src.cli file-demo
python -m unittest discover -s tests -v
```

Lampiran B. Kode Sumber src/aesgcm_lab.py

```
"""aesgcm_lab
modul praktikum AES-256-GCM (Project 1 Kriptografi).
Penyusun: Abdullah Wahid
Seluruh operasi kriptografi dikerjakan oleh pustaka `cryptography`.
Modul ini hanya membungkusnya menjadi antarmuka yang lebih mudah diuji:
enkripsi teks/byte, enkripsi berkas dengan format wadah sederhana, serta
fungsi demonstrasi risiko mode ECB. Hanya untuk pembelajaran.
"""
from     future     import annotations

import os
from dataclasses import dataclass
from pathlib import Path

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

KEY_LEN = 32        # byte -> AES-256
NONCE  LEN = 12      # byte -> 96 bit (rekomendasi NIST SP 800-38D)
TAG_LEN = 16        # byte -> 128 bit
MAGIC = b"VGCM1"    # penanda format berkas terenkripsi (juga dipakai sebagai AAD)
BLOCK = 16

class ContainerError(ValueError):
    """Berkas terenkripsi rusak, terpotong, atau bukan format yang dikenal."""

@dataclass(frozen=True)
class Sealed:
    """Hasil enkripsi GCM yang dipisah menjadi tiga komponen."""
    nonce: bytes
    ciphertext: bytes
    tag: bytes

def new  key() -> bytes:
    return AESGCM.generate_key(bit_length=KEY_LEN * 8)

def new_nonce() -> bytes:
    return os.urandom(NONCE_LEN)

def seal(key: bytes, data: bytes, aad: bytes = b"", nonce: bytes | None = None) -> Sealed:
```

```
    """Enkripsi `data`. Nonce acak baru dibuat bila tidak diberikan."""
    nonce = new_nonce() if nonce is None else nonce
    if len(nonce) != NONCE_LEN:
        raise ValueError(f"Nonce harus {NONCE_LEN} byte.")
    out = AESGCM(key).encrypt(nonce, data, aad or None)
    return Sealed(nonce, out[:-TAG_LEN], out[-TAG_LEN:])

def unseal(key: bytes, sealed: Sealed, aad: bytes = b"") -> bytes:
    """Verifikasi tag lalu dekripsi. Melempar InvalidTag bila autentikasi gagal."""
    if len(sealed.nonce) != NONCE_LEN:
        raise ValueError(f"Nonce harus {NONCE_LEN} byte.")
    if len(sealed.tag) != TAG_LEN:
        raise ValueError(f"Tag harus {TAG_LEN} byte.")
    return AESGCM(key).decrypt(sealed.nonce, sealed.ciphertext + sealed.tag, aad or None)

def seal_text(key: bytes, text: str, aad: bytes = b"") -> Sealed:
    return seal(key, text.encode("utf-8"), aad)

def unseal_text(key: bytes, sealed: Sealed, aad: bytes = b"") -> str:
    return unseal(key, sealed, aad).decode("utf-8")

# --- enkripsi berkas ---
# Format: MAGIC (5) | nonce (12) | ciphertext (n) | tag (16)

def encrypt_file(src: Path, dst: Path, key: bytes) -> int:
    s = seal(key, Path(src).read_bytes(), aad=MAGIC)
    blob = MAGIC + s.nonce + s.ciphertext + s.tag
    Path(dst).write_bytes(blob)
    return len(blob)

def decrypt_file(src: Path, dst: Path, key: bytes) -> int:
    blob = Path(src).read_bytes()
    if len(blob) < len(MAGIC) + NONCE_LEN + TAG_LEN:
        raise ContainerError("Berkas terlalu pendek untuk format VGCM1.")
    if blob[: len(MAGIC)] != MAGIC:
        raise ContainerError("Penanda format tidak dikenali.")

    body = blob[len(MAGIC):]
    sealed = Sealed(body[:NONCE_LEN], body[NONCE_LEN:-TAG_LEN], body[-TAG_LEN:])
    plain = unseal(key, sealed, aad=MAGIC)
    # tulis ke disk hanya jika tag valid
    Path(dst).write_bytes(plain)
    return len(plain)

# --- demonstrasi ECB (hanya untuk laporan) ---

def ecb_encrypt_for_demo(key: bytes, data: bytes) -> bytes:
    """JANGAN dipakai untuk data nyata. Dipakai hanya untuk memperlihatkan pola ECB."""
    if len(data) % BLOCK:
        raise ValueError("Data demo harus kelipatan 16 byte.")
    enc = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return enc.update(data) + enc.finalize()

def split_blocks(data: bytes) -> list[bytes]:
    return [data[i:i + BLOCK] for i in range(0, len(data), BLOCK)]

def count_repeated_blocks(data: bytes) -> int:
    blocks = split_blocks(data)
    return len(blocks) - len(set(blocks))
```

Lampiran C. Kode Sumber src/cli.py

```
"""Antarmuka baris perintah praktikum AES-256-GCM.
Penyusun: Abdullah Wahid

Contoh:
  python -m src.cli demo
  python -m src.cli keygen --out rahasia.key
  python -m src.cli encrypt-file --key rahasia.key --in catatan.txt --out catatan.vgcm
  python -m src.cli decrypt-file --key rahasia.key --in catatan.vgcm --out hasil.txt
  python -m src.cli ecb-demo
"""
from __future__ import annotations

import argparse
import hashlib
import os
import sys
from pathlib import Path

from cryptography.exceptions import InvalidTag

import src.aesgcm_lab as lab

PLAINTEXT = "ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI"
AAD_DEMO = b"kriptografi-project1"
LINE = "-" * 66

def hx(b: bytes) -> str:
    return b.hex().upper()

def cmd_demo(_: argparse.Namespace) -> int:
    key = lab.new_key()
    print("=" * 66)
    print(" AES-256-GCM | Praktikum Kriptografi | Abdullah Wahid")
    print("=" * 66)
    print(f"Plaintext      : {PLAINTEXT}")
    print(f"Plaintext (hex): {hx(PLAINTEXT.encode())}")
    print(f"Kunci          : {len(key)*8} bit (nilai tidak ditampilkan)")
    print(f"AAD            : {AAD_DEMO.decode()}")

    s = lab.seal  text(key, PLAINTEXT, AAD  DEMO)
    print(LINE)
    print(f"Nonce          : {hx(s.nonce)}")
    print(f"Ciphertext     : {hx(s.ciphertext)}")
    print(f"Tag            : {hx(s.tag)}")
    print(LINE)

    back = lab.unseal_text(key, s, AAD_DEMO)
    print(f"Hasil dekripsi : {back}")
    ok = back == PLAINTEXT
    print(f"Verifikasi     : {'COCOK - SUKSES' if ok else 'TIDAK COCOK - GAGAL'}")

    try:
        lab.unseal_text(key, s, b"aad-yang-salah")
    except InvalidTag:
        print("Uji AAD salah  : DITOLAK (InvalidTag)")
    return 0 if ok else 1

def cmd_keygen(a: argparse.Namespace) -> int:
    out = Path(a.out)
```

```
    if out.exists():
        print(f"Menolak menimpa {out}.", file=sys.stderr)
        return 1
    out.write_text(lab.new_key().hex())
    try:
        os.chmod(out, 0o600)
    except OSError:
        pass
    print(f"Kunci 256 bit ditulis ke {out}. Jangan commit/bagikan berkas ini.")
    return 0

def load_key(path: str) -> bytes:
    key = bytes.fromhex(Path(path).read_text().strip())
    if len(key) != lab.KEY_LEN:
        raise SystemExit("Panjang kunci harus 32 byte (64 karakter hex).")
    return key

def sha(path: Path) -> str:
    return hashlib.sha256(path.read_bytes()).hexdigest()[:16].upper()

def cmd_encrypt(a: argparse.Namespace) -> int:
    n = lab.encrypt_file(Path(a.infile), Path(a.out), load_key(a.key))
    print(f"Terenkripsi: {a.infile} -> {a.out} ({n} byte)")
    return 0

def cmd_decrypt(a: argparse.Namespace) -> int:
    try:
        n = lab.decrypt_file(Path(a.infile), Path(a.out), load_key(a.key))
    except InvalidTag:
        print("GAGAL: tag tidak valid (kunci salah atau berkas diubah).", file=sys.stderr)
        return 2
    except lab.ContainerError as e:
        print(f"GAGAL: {e}", file=sys.stderr)
        return 3
    print(f"Terdekripsi: {a.infile} -> {a.out} ({n} byte)")
    return 0

def cmd_ecb(_: argparse.Namespace) -> int:
    key = lab.new_key()
    data = b"BLOK-UJI-AES-128" * 4  # empat blok 16 byte yang identik
    ecb = lab.ecb_encrypt_for_demo(key, data)
    s = lab.seal(key, data)

    print("Plaintext: 4 blok identik (16 byte x 4)")
    print(LINE)
    print("ECB (jangan dipakai):")
    for i, b in enumerate(lab.split_blocks(ecb), 1):
        print(f"  blok {i}: {hx(b)}")
    print(f"  blok berulang : {lab.count_repeated_blocks(ecb)} -> pola plaintext terlihat")

    print("GCM:")
    for i, b in enumerate(lab.split_blocks(s.ciphertext), 1):
        print(f"  blok {i}: {hx(b)}")
    print(f"  blok berulang : {lab.count_repeated_blocks(s.ciphertext)} -> pola tersamarkan")
    return 0

def cmd_filedemo(_: argparse.Namespace) -> int:
    import tempfile
    key = lab.new_key()
    with tempfile.TemporaryDirectory() as d:
        d = Path(d)
```

```
        src, enc, dec = d / "catatan.txt", d / "catatan.vgcm", d / "hasil.txt"
        src.write_text("Catatan uji untuk enkripsi berkas.\n" * 20)

        lab.encrypt_file(src, enc, key)
        lab.decrypt_file(enc, dec, key)
        blob = enc.read_bytes()

        print(f"Berkas asli      : {src.stat().st_size} byte  SHA-256[:16]={sha(src)}")
        print(f"Berkas terenkrip : {enc.stat().st_size} byte  (= asli + 5 + 12 + 16)")
        print(f"Header (hex)     : {hx(blob[:5])} | nonce {hx(blob[5:17])}")
        print(f"Hasil dekripsi   : {dec.stat().st_size} byte  SHA-256[:16]={sha(dec)}")
        print(f"Hash sama        : {sha(src) == sha(dec)}")

        bad = bytearray(blob)
        bad[20] ^= 0x01
        enc.write_bytes(bytes(bad))
        try:
            lab.decrypt_file(enc, dec, key)
        except InvalidTag:
            print("Berkas diubah 1 bit -> DITOLAK (InvalidTag), tidak ada keluaran ditulis")
    return 0

def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
        prog="python -m src.cli",
        description=__doc__,
        formatter_class=argparse.RawDescriptionHelpFormatter
    )
    sub = p.add_subparsers(dest="cmd", required=True)
    sub.add_parser("demo", help="demo enkripsi teks + AAD").set_defaults(fn=cmd_demo)

    k = sub.add_parser("keygen", help="buat berkas kunci 256 bit")
    k.add_argument("--out", required=True)
    k.set_defaults(fn=cmd_keygen)

    for name, fn in (("encrypt-file", cmd_encrypt), ("decrypt-file", cmd_decrypt)):
        e = sub.add_parser(name)
        e.add_argument("--key", required=True)
        e.add_argument("--in", dest="infile", required=True)
        e.add_argument("--out", required=True)
        e.set_defaults(fn=fn)

    sub.add_parser("ecb-demo", help="bandingkan ECB dan GCM").set_defaults(fn=cmd_ecb)
    sub.add_parser("file-demo", help="demo enkripsi berkas sementara").set_defaults(fn=cmd_filedemo)
    return p

def main(argv: list[str] | None = None) -> int:
    args = build_parser().parse_args(argv)
    return args.fn(args)

if __name__ == "__main__":
    raise SystemExit(main())
```

Lampiran D. Kode Sumber tests/test_aesgcm_lab.py

```
import tempfile
import unittest
from pathlib import Path
```

```
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

import src.aesgcm_lab as lab

TEKS = "ABDULLAH WAHID - PROJECT 1 KRIPTOGRAFI"

def flip(b: bytes, i: int = 0) -> bytes:
    x = bytearray(b)
    x[i] ^= 0x01
    return bytes(x)

class KnownAnswerTests(unittest.TestCase):
    """Vektor uji resmi, agar kebenaran pustaka dan parameter terverifikasi."""

    def test_fips197_c3_aes256_block(self):
        key = bytes(range(32))
        pt = bytes.fromhex("00112233445566778899aabbccddeeff")
        enc = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
        self.assertEqual(
            (enc.update(pt) + enc.finalize()).hex(),
            "8ea2b7ca516745bfeafc49904b496089"
        )

    def test_gcm_aes256_zero_key_empty_plaintext(self):
        s = lab.seal(bytes(32), b"", nonce=bytes(12))
        self.assertEqual(s.ciphertext, b"")
        self.assertEqual(s.tag.hex(), "530f8afbc74536b9a963b4f1c4cb738b")

    def test_gcm_aes256_zero_key_one_zero_block(self):
        s = lab.seal(bytes(32), bytes(16), nonce=bytes(12))
        self.assertEqual(s.ciphertext.hex(), "cea7403d4d606b6e074ec5d3baf39d18")
        self.assertEqual(s.tag.hex(), "d0d1c8a799996bf0265b98b5d48ab919")

class RoundTripAndTamper(unittest.TestCase):

    def setUp(self):
        self.key = lab.new_key()
        self.aad = b"header-uji"
        self.s = lab.seal_text(self.key, TEKS, self.aad)

    def test_roundtrip_text(self):
        self.assertEqual(lab.unseal_text(self.key, self.s, self.aad), TEKS)

    def test_ciphertext_same_length_as_plaintext(self):
        self.assertEqual(len(self.s.ciphertext), len(TEKS.encode()))

    def test_flip_every_ciphertext_byte_is_rejected(self):
        for i in range(len(self.s.ciphertext)):
            bad = lab.Sealed(self.s.nonce, flip(self.s.ciphertext, i), self.s.tag)
            with self.assertRaises(InvalidTag, msg=f"byte {i}"):
                lab.unseal(self.key, bad, self.aad)

    def test_flip_every_tag_byte_is_rejected(self):
        for i in range(lab.TAG_LEN):
            bad = lab.Sealed(self.s.nonce, self.s.ciphertext, flip(self.s.tag, i))
            with self.assertRaises(InvalidTag):
                lab.unseal(self.key, bad, self.aad)

    def test_wrong_key(self):
```

```
        with self.assertRaises(InvalidTag):
            lab.unseal(lab.new_key(), self.s, self.aad)

    def test_wrong_nonce(self):
        bad = lab.Sealed(flip(self.s.nonce), self.s.ciphertext, self.s.tag)
        with self.assertRaises(InvalidTag):
            lab.unseal(self.key, bad, self.aad)

    def test_wrong_aad(self):
        with self.assertRaises(InvalidTag):
            lab.unseal(self.key, self.s, b"aad-lain")

    def test_bad_lengths(self):
        with self.assertRaises(ValueError):
            lab.seal(self.key, b"x", nonce=b"pendek")
        with self.assertRaises(ValueError):
            lab.unseal(self.key, lab.Sealed(self.s.nonce, self.s.ciphertext, b"x"))

    def test_nonces_do_not_repeat_in_10000_draws(self):
        self.assertEqual(len({lab.new_nonce() for _ in range(10000)}), 10000)

    def test_same_plaintext_gives_different_ciphertext(self):
        a = lab.seal_text(self.key, TEKS)
        b = lab.seal_text(self.key, TEKS)
        self.assertNotEqual(a.ciphertext, b.ciphertext)

class FileContainer(unittest.TestCase):

    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.d = Path(self.tmp.name)
        self.key = lab.new_key()
        self.src = self.d / "a.bin"
        self.src.write_bytes(bytes(range(256)) * 10)
        self.enc = self.d / "a.vgcm"
        self.out = self.d / "b.bin"

    def tearDown(self):
        self.tmp.cleanup()

    def test_file_roundtrip(self):
        n = lab.encrypt_file(self.src, self.enc, self.key)
        self.assertEqual(n, 2560 + len(lab.MAGIC) + lab.NONCE_LEN + lab.TAG_LEN)
        lab.decrypt_file(self.enc, self.out, self.key)
        self.assertEqual(self.out.read_bytes(), self.src.read_bytes())

    def text_modified_file_is_rejected_and_nothing_written(self):
        lab.encrypt_file(self.src, self.enc, self.key)
        self.enc.write_bytes(flip(self.enc.read_bytes(), 30))
        with self.assertRaises(InvalidTag):
            lab.decrypt_file(self.enc, self.out, self.key)
        self.assertFalse(self.out.exists())

    def test_truncated_and_wrong_magic(self):
        self.enc.write_bytes(b"VGCM1abc")
        with self.assertRaises(lab.ContainerError):
            lab.decrypt_file(self.enc, self.out, self.key)
        self.enc.write_bytes(b"XXXXX" + bytes(40))
        with self.assertRaises(lab.ContainerError):
            lab.decrypt_file(self.enc, self.out, self.key)
```

```
class EcbDemo(unittest.TestCase):

    def test_ecb_leaks_repetition_but_gcm_does_not(self):
        key = lab.new_key()
        data = b"0123456789ABCDEF" * 4
        self.assertEqual(lab.count_repeated_blocks(lab.ecb_encrypt_for_demo(key, data)), 3)
        self.assertEqual(lab.count_repeated_blocks(lab.seal(key, data).ciphertext), 0)

if __name__ == "__main__":
    unittest.main()
```